

PRACTICAL TUTORIAL

CI/CD with GitHub Actions & GitHub Self-hosted Runner

Summary

This tutorial provides step-by-step guidance on setting up a complete CI/CD pipeline using GitHub Actions, including installing a GitHub Self-hosted Runner on your personal computer and building a real Python demo project to illustrate the entire automation workflow from code commit to deployment.

Upon completion, students will be able to:

- (1) understand CI/CD principles,
- (2) configure GitHub Actions workflows,
- (3) operate a GitHub Runner locally, and
- (4) apply these skills to real projects.

1. OVERVIEW OF CI/CD

1.1. What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. It is a set of modern software development practices that automate the process of building, testing, and deploying applications.

Concept	Meaning	Primary Benefit
CI – Continuous Integration	Automatically build and test every time a developer pushes code to the repository	Early bug detection, preventing bug accumulation
CD – Continuous Delivery	Ensure code is always ready to be deployed to production at any time	Shorter release cycles, higher software quality
CD – Continuous Deployment	Automatically deploy code to production immediately after all tests pass	Zero manual intervention, maximum deployment speed

1.2. Why is CI/CD Important?

Before CI/CD became widespread, development teams frequently experienced 'Integration Hell' – a situation where multiple developers worked in parallel for extended periods before attempting to merge code, resulting in countless conflicts and difficult-to-resolve errors. CI/CD addresses this problem through continuous integration, detecting and fixing errors early.

Real-World Example: VNPAY and CI/CD

VNPAY – Vietnam's largest electronic payment gateway – processes millions of transactions daily. Their system demands zero downtime and absolute stability.

Before CI/CD: Each new feature release required 2–3 weeks of manual testing. Developers were reluctant to fix minor bugs for fear of affecting production.

After CI/CD: Release cycles shortened to a few hours. The team could deploy 10–20 times per day. Bugs were detected within minutes of pushing code. The QA team shifted focus to complex business testing instead of manual regression testing.

1.3. GitHub Actions – GitHub's CI/CD Platform

GitHub Actions is a CI/CD service built into GitHub that allows you to automate workflows directly within your repository. Key highlights:

- Free for public repositories and 2,000 minutes/month for private repositories.
- Natively integrated with every GitHub repository – no additional CI server needed.
- Rich Marketplace with thousands of ready-made Actions available.
- Supports Self-hosted Runners – run on your own machine with no minute limits.

2. KEY CONCEPTS IN GITHUB ACTIONS

2.1. Basic Structure

Before starting the hands-on exercises, you need to understand the core concepts of GitHub Actions:

Term	Definition	Real-World Example
Workflow	A set of automated jobs defined in a YAML file	ci.yml, deploy.yml, test.yml
Event	An action that triggers the workflow to run	push, pull_request, schedule, workflow_dispatch
Job	A set of steps running on the same runner	job: build, job: test, job: deploy
Step	A single task within a job (run a command or Action)	Run pytest, Run docker build
Action	A reusable unit, typically from GitHub Marketplace	actions/checkout@v4, actions/setup-python@v5
Runner	The server that executes jobs (GitHub-hosted or self-hosted)	ubuntu-latest, windows-latest, self-hosted
Artifact	Files saved after a workflow run completes	Test reports, build outputs, logs

2.2. YAML Workflow File Structure

Every GitHub Actions workflow is defined in a YAML file located in the `.github/workflows/` directory of your repository. Below is the basic structure with explanations for each part:

```
# Workflow name -- appears in the Actions tab of GitHub
name: CI Pipeline Demo

# TRIGGERS: Events that activate the workflow
on:
  push:                                # When code is pushed
    branches: [ main, dev ]           # Only for main and dev branches
  pull_request:                        # When a Pull Request is created
    branches: [ main ]
  workflow_dispatch:                  # Allow manual run from GitHub UI

# JOBS: Tasks to be executed
jobs:
  build-and-test:                     # Job name (customizable)
    runs-on: self-hosted              # Runner: use your local machine
    steps:                             # Steps within the job
      - name: Checkout code           # Step 1: Fetch code from repo
```

```

    uses: actions/checkout@v4
  - name: Set up Python      # Step 2: Install Python
    uses: actions/setup-python@v5
    with:
      python-version: '3.11'
  - name: Install deps      # Step 3: Install dependencies
    run: pip install -r requirements.txt
  - name: Run tests         # Step 4: Execute tests
    run: pytest tests/ -v

```

2.3. GitHub-hosted Runner vs Self-hosted Runner

GitHub provides two types of runners with different characteristics and use cases:

Criterion	GitHub-hosted Runner	Self-hosted Runner
Infrastructure	GitHub's servers	Your own machine
Cost	Free with limits (2,000 min/month)	Completely free, unlimited
Configuration	Standard (2 CPU, 7GB RAM)	Your machine's specs
Internal network access	No (isolated network)	Yes – can access localhost, internal DBs
Security	Clean environment per run	You manage runner security
Best suited for	Open source, simple CI	Private infra, ML model training, learning demos
YAML syntax	runs-on: ubuntu-latest	runs-on: self-hosted

3. INSTALLING GITHUB SELF-HOSTED RUNNER LOCALLY

3.1. System Requirements

Prerequisites

- Operating System: Windows 10/11, macOS 10.15+, or Ubuntu 20.04+ (Linux).
- GitHub Account: Registered and logged in. Create one free at <https://github.com>.
- Git: Installed on your machine. Verify with: `git --version`.
- Python 3.8+: Installed. Verify with: `python --version` or `python3 --version`.
- Internet Connection: Runner needs to connect to GitHub to receive jobs.
- Admin Privileges: Required to install the runner service (especially on Windows/Linux).

3.2. Creating a Repository on GitHub

**Step
1**

Log in to GitHub and create a new repository

Go to <https://github.com>, click the '+' button in the top-right corner, select 'New repository'.

**Step
2**

Fill in the repository details

Repository name: github-runner-demo | Description: Demo CI/CD with GitHub Self-hosted Runner | Choose Public | Check 'Add a README file' | Click 'Create repository'.

**Step
3**

Clone the repository to your local machine

Copy the repository URL, open Terminal/Command Prompt and run the clone command below.

```
# Clone the repository (replace YOUR_USERNAME with your username)
git clone https://github.com/YOUR_USERNAME/github-runner-demo.git

# Navigate into the project directory
cd github-runner-demo

# Check status
git status
```

3.3. Installing the Self-hosted Runner – Detailed Guide

GitHub provides an automated runner installation script. Follow the steps below for your operating system:

3.3.1. Obtaining the Installation Script from GitHub

Go to the repository you just created on GitHub and follow these steps:

- 1. Go to the repository's Settings tab (gear icon).
- 2. In the left sidebar, select Actions → Runners.
- 3. Click the green New self-hosted runner button.
- 4. Select your operating system: Linux, macOS, or Windows.

Important Note

GitHub generates a special token (Registration Token) for each installation. This token is valid for only 1 hour. Complete the setup within this timeframe. If it expires, generate a new token from Settings → Actions → Runners.

3.3.2. Installation on Linux / macOS

```
# STEP 1: Create a directory for the runner
mkdir actions-runner && cd actions-runner

#STEP 2: Download the runner package
# For Linux x64:
curl -o actions-runner-linux-x64-2.319.1.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.319.1/actions-runner-linux-
x64-2.319.1.tar.gz

# For macOS x64:
curl -o actions-runner-osx-x64-2.319.1.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.319.1/actions-runner-osx-
x64-2.319.1.tar.gz

# For macOS ARM (Apple M1/M2/M3/M4/M5):
curl -o actions-runner-osx-arm64-2.319.1.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.319.1/actions-runner-osx-
arm64-2.319.1.tar.gz

# STEP 3: Extract the archive
tar xzf ./actions-runner-linux-x64-2.319.1.tar.gz

# STEP 4: Configure the runner (paste the command from GitHub)
./config.sh --url https://github.com/YOUR_USERNAME/github-runner-demo \
  --token YOUR_REGISTRATION_TOKEN

# When prompted, enter:
# Runner group: [Press Enter for Default]
# Runner name: my-local-runner [or any name you prefer]
# Additional labels: [Press Enter to skip]
# Work folder: [Press Enter for default _work]

# STEP 5: Start the runner
./run.sh

# Expected output:
# Connected to GitHub
# 2024-01-15 10:30:00Z: Listening for Jobs
```

3.3.3. Installation on Windows

```
# Open PowerShell as Administrator
# (right-click Start → Windows PowerShell (Admin))

# STEP 1: Create directory
mkdir C:\actions-runner; cd C:\actions-runner

# STEP 2: Download runner
```

```

Invoke-WebRequest -Uri
https://github.com/actions/runner/releases/download/v2.319.1/actions-runner-win-x64-
2.319.1.zip -OutFile actions-runner-win-x64-2.319.1.zip

# STEP 3: Extract
Add-Type -AssemblyName System.IO.Compression.FileSystem;
[System.IO.Compression.ZipFile]::ExtractToDirectory('$PWD/actions-runner-win-x64-
2.319.1.zip', '$PWD')

# STEP 4: Configure (paste command from GitHub)
./config.cmd --url https://github.com/YOUR_USERNAME/github-runner-demo `
--token YOUR_REGISTRATION_TOKEN

# STEP 5: Run the runner
./run.cmd

```

3.3.4. Running the Runner as a Background Service (Optional)

To have the runner automatically restart when your computer reboots, install it as a system service:

```

# Linux: Install as systemd service
sudo ./svc.sh install
sudo ./svc.sh start

# Check status:
sudo ./svc.sh status

# Stop service:
sudo ./svc.sh stop

# macOS: Install as launchd service
./svc.sh install
./svc.sh start

# Windows: Runner installs as Windows Service automatically
# After running config.cmd, additionally run:
./svc.sh install
./svc.sh start

```

3.4. Verifying the Runner is Connected

After running `./run.sh` (Linux/macOS) or `./run.cmd` (Windows), you need to confirm that the runner has successfully connected to GitHub:

**Step
1**

Check Terminal/Command Prompt

The terminal will display: *Connected to GitHub* and Listening for Jobs. This confirms the runner is operational.

**Step
2****Check on GitHub**

Return to Settings → Actions → Runners in your repository. Your runner will appear with a green status: Idle (waiting for jobs).

Expected Result

On GitHub, in the Settings → Actions → Runners page, you should see your runner with:

- Name: my-local-runner (or the name you assigned)
- Status: Idle (green)
- Labels: self-hosted, Linux/macOS/Windows (depending on your OS)
- Last seen: a few seconds ago

4. CREATING THE PYTHON DEMO PROJECT

4.1. Project Overview

We will build a simple computation application – a Calculator Library – containing basic mathematical functions and a comprehensive unit test suite. This project is designed to clearly illustrate the entire CI pipeline: code → test → report.

Project Directory Structure	File Descriptions
github-runner-demo/	Project root directory
├── .github/	GitHub configuration folder
├── workflows/	Contains YAML workflow files
└── ci.yml	Main CI pipeline definition file
├── src/	Application source code
├── __init__.py	Marks src as a Python package
└── calculator.py	Main computation module
├── tests/	Unit test directory
├── __init__.py	Marks tests as a Python package
└── test_calculator.py	Complete unit test file
└── requirements.txt	Python dependencies list
└── README.md	Project documentation

4.2. Creating the Directory Structure

Open Terminal/Command Prompt, ensure you are inside the *github-runner-demo* directory, then run the following commands:

```
# Ensure you are in the project directory
cd github-runner-demo

# Create the directory structure
mkdir -p src tests .github/workflows

# Create __init__.py files
touch src/__init__.py
touch tests/__init__.py

# Verify the created structure
find . -type f -name '*.py'
ls -la
```

4.3. Writing the Application Code

4.3.1. File: src/calculator.py

This is the main module of the application. Create the file `src/calculator.py` with the following content:

```
'''
calculator.py -- Basic computation module
CI/CD demo project with GitHub Actions
'''

class Calculator:
    '''Class performing basic mathematical operations.'''

    def add(self, a: float, b: float) -> float:
        '''Add two numbers.'''
        return a + b

    def subtract(self, a: float, b: float) -> float:
        '''Subtract b from a.'''
        return a - b

    def multiply(self, a: float, b: float) -> float:
        '''Multiply two numbers.'''
        return a * b

    def divide(self, a: float, b: float) -> float:
        '''Divide a by b. Raises ValueError if b = 0.'''
        if b == 0:
            raise ValueError('Cannot divide by zero!')
        return a / b
```

```

def power(self, base: float, exponent: float) -> float:
    '''Calculate base raised to the power of exponent.'''
    return base ** exponent

def is_positive(self, n: float) -> bool:
    '''Check if a number is positive.'''
    return n > 0

```

4.3.2. File: tests/test_calculator.py

Create a comprehensive unit test suite. Create the file tests/test_calculator.py with the following content:

```

import pytest
from src.calculator import Calculator

# Shared Calculator instance for all tests
@pytest.fixture
def calc():
    return Calculator()

# Addition tests
class TestAddition:
    def test_add_two_positive_numbers(self, calc):
        assert calc.add(3, 5) == 8

    def test_add_negative_numbers(self, calc):
        assert calc.add(-3, -5) == -8

    def test_add_positive_and_negative(self, calc):
        assert calc.add(10, -3) == 7

    def test_add_with_zero(self, calc):
        assert calc.add(5, 0) == 5

    def test_add_floats(self, calc):
        assert abs(calc.add(1.5, 2.3) - 3.8) < 0.0001

# Subtraction tests
class TestSubtraction:
    def test_subtract_basic(self, calc):
        assert calc.subtract(10, 4) == 6

    def test_subtract_resulting_negative(self, calc):
        assert calc.subtract(3, 10) == -7

# Multiplication tests
class TestMultiplication:

```

```

def test_multiply_basic(self, calc):
    assert calc.multiply(4, 5) == 20

def test_multiply_by_zero(self, calc):
    assert calc.multiply(100, 0) == 0

def test_multiply_negatives(self, calc):
    assert calc.multiply(-3, -4) == 12

# Division tests
class TestDivision:
    def test_divide_basic(self, calc):
        assert calc.divide(10, 2) == 5

    def test_divide_resulting_float(self, calc):
        assert calc.divide(7, 2) == 3.5

    def test_divide_by_zero_raises_error(self, calc):
        with pytest.raises(ValueError, match='Cannot divide by zero'):
            calc.divide(10, 0)

# Other operation tests
class TestOtherOperations:
    def test_power_basic(self, calc):
        assert calc.power(2, 10) == 1024

    def test_power_zero_exponent(self, calc):
        assert calc.power(999, 0) == 1

    def test_is_positive_true(self, calc):
        assert calc.is_positive(5) == True

    def test_is_positive_false(self, calc):
        assert calc.is_positive(-1) == False

    def test_is_positive_zero(self, calc):
        assert calc.is_positive(0) == False

```

4.3.3. File: requirements.txt

Create the requirements.txt file at the project root with the following content:

```

# Testing framework
pytest==7.4.4

# Test coverage reporting
pytest-cov==4.1.0

```

Tip: Test Locally Before Pushing

Before pushing to GitHub, run tests locally to ensure there are no basic errors:

```
pip install -r requirements.txt
```

```
pytest tests/ -v
```

If all tests PASS, the code is ready for the CI pipeline to process.

5. CONFIGURING THE GITHUB ACTIONS WORKFLOW

5.1. Creating the CI/CD Workflow File

Create the file `.github/workflows/ci.yml` with the content below. This is the heart of the entire CI pipeline:

```
# CI Pipeline -- github-runner-demo
# Uses GitHub Self-hosted Runner for automated build and testing

name: CI Pipeline with Self-hosted Runner

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch: # Allow manual run from GitHub UI

jobs:
  # JOB 1: Code quality check
  lint-and-format:
    name: Code Quality Check
    runs-on: self-hosted
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install linting tools
        run: |
          python -m pip install --upgrade pip
          pip install flake8

      - name: Run flake8 linter
```

```

    run: |
        # Check for syntax errors and code style
        # --max-line-length=120: allows lines up to 120 characters
        flake8 src/ tests/ --max-line-length=120 --statistics

# JOB 2: Run Unit Tests
run-tests:
  name: Run Unit Tests
  runs-on: self-hosted
  needs: lint-and-format # Only runs after lint succeeds
  steps:
    - name: Checkout source code
      uses: actions/checkout@v4

    - name: Set up Python 3.11
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt

    - name: Run tests with coverage report
      run: |
        pytest tests/ -v \
          --cov=src \
          --cov-report=term-missing \
          --cov-report=xml:coverage.xml

    - name: Upload coverage report
      uses: actions/upload-artifact@v4
      if: always() # Upload even if tests fail
      with:
        name: coverage-report
        path: coverage.xml

# JOB 3: Build Summary
build-summary:
  name: Build Summary
  runs-on: self-hosted
  needs: [lint-and-format, run-tests]
  if: always() # Always runs to summarize results
  steps:
    - name: Print success message
      if: needs.run-tests.result == 'success'
      run: |
        echo '=====
        echo ' CI PIPELINE PASSED!'

```

```

    echo ' Ready to deploy to production.'
    echo '===== '

- name: Print failure message
  if: needs.run-tests.result == 'failure'
  run: |
    echo ' CI PIPELINE FAILED!'
    echo ' Check logs to identify errors.'

```

5.2. Detailed Explanation of the Workflow File

Let us analyze each important section of the workflow file above:

5.2.1. Triggers Section (on:)

This section defines when the workflow is activated:

- `push: branches: [main, develop]` – Runs when code is pushed to the main or develop branch.
- `pull_request: branches: [main]` – Runs when a Pull Request targeting the main branch is created. Critical for validating code before merging.
- `workflow_dispatch` – Allows manually running the workflow from the Actions tab on GitHub. Useful for debugging or re-running the pipeline.

5.2.2. The `needs:` Keyword – Controlling Execution Order

The `needs:` keyword creates dependencies between jobs, ensuring a job only runs after another completes successfully:

```

run-tests:
  needs: lint-and-format # Only run tests if lint PASSES

build-summary:
  needs: [lint-and-format, run-tests] # Wait for both jobs to complete
  if: always() # Run even if other jobs fail (to report results)

```

Why Split into Multiple Jobs?

Dividing the workflow into multiple jobs offers several advantages:

- **Earlier error detection:** If lint fails, there is no wasted time running tests.
- **Parallelization:** Jobs without dependencies can run simultaneously.
- **Clear reporting:** In the GitHub UI, you can immediately see which job failed.
- **Reusability:** Each job can use a different Runner as needed.

6. RUNNING AND MONITORING THE CI PIPELINE

6.1. Pushing Code to GitHub

Now we will push all the code to GitHub and watch the CI pipeline run automatically:

**Step
1**

Check the status of created files

Before committing, verify that all files were created correctly.

```
# Check git status
git status

# Expected output - you should see all new files:
# Untracked files:
#   .github/workflows/ci.yml
#   src/__init__.py
#   src/calculator.py
#   tests/__init__.py
#   tests/test_calculator.py
#   requirements.txt
```

**Step
2**

Stage all files and create a commit

Add all files to the staging area and create a commit with a meaningful message.

```
# Add all files to staging
git add .

# Create a descriptive commit message
git commit -m 'feat: add calculator module with unit tests and CI pipeline'

# Confirm the commit was created
git log --oneline -3
```

**Step
3**

Push code to GitHub

Push code to the remote repository. The pipeline will automatically trigger immediately after the push completes.

```
# Push to the main branch
git push origin main

# If this is your first push, you may need:
git push -u origin main
```

6.2. Monitoring the Pipeline on GitHub

After pushing, the pipeline will automatically start within seconds. Follow these steps to monitor it:

Step 1

Access the Actions tab

Go to the repository on GitHub and click the Actions tab (circular arrow icon). You will see the recently triggered workflow.

Step 2

Click on the workflow run for details

Click the workflow run name (e.g., feat: add calculator module...). You will see a diagram of jobs with color-coded statuses.

Step 3

View logs for each step

Click into each Job, then expand each Step to view detailed logs. This is how you debug when a pipeline fails.

Step 4

Monitor the Runner on Terminal

On the local machine where the Runner is running, the Terminal will display real-time output as jobs are received and executed.

6.3. Understanding Pipeline Results

Icon	Status	Meaning and Action Required
 Spinning yellow circle	In Progress	Workflow is running. Runner has received the job and is executing. Wait for completion.
 Green checkmark	Success	All jobs PASSED. Code is clean, all tests pass. Ready to merge PR or deploy.
 Red X	Failure	At least one job FAILED. Click to view error logs, fix the code, and push again.
 Gray circle	Skipped	Job was skipped because the condition (if:) was not met, or a dependency job failed.
 Blue dot	Queued	Job is waiting for a Runner to become available. If waiting too long, check if the Runner is active.

7. TESTING SCENARIOS AND DEBUGGING SKILLS

7.1. Scenario 1 – Successful Pipeline (Happy Path)

This is the ideal case: all code is correct and all tests pass. You completed this in Section 6. Observe the log output to understand how the pipeline behaves when everything works correctly.

Expected pytest Output in Logs

```
===== test session starts =====  
collected 18 items  
tests/test_calculator.py::TestAddition::test_add_two_positive_numbers PASSED  
tests/test_calculator.py::TestAddition::test_add_negative_numbers PASSED  
tests/test_calculator.py::TestDivision::test_divide_by_zero_raises_error PASSED  
... (15 more tests)  
===== 18 passed in 0.52s =====
```

7.2. Scenario 2 – Intentionally Introducing a Bug to See Pipeline Fail

This exercise helps you understand the CI error detection mechanism. We will introduce a bug in the code and observe how the pipeline reports the error:

**Step
1**

Introduce a bug in calculator.py

Open `src/calculator.py` and modify the `add()` function to return an incorrect result (`a - b` instead of `a + b`).

```
# Change in src/calculator.py  
def add(self, a: float, b: float) -> float:  
    '''Add two numbers.'''  
    return a - b # BUG: wrong operation! Should be a + b
```

**Step
2**

Commit and push the buggy code

Push the code to GitHub to trigger the pipeline.

```
git add src/calculator.py  
git commit -m 'bug: introduce intentional bug for CI demo'  
git push origin main
```

**Step
3**

Observe the pipeline on GitHub

Go to the Actions tab. The `run-tests` job will fail and display in red. Click to view the pytest error logs.

**Step
4**

Read and understand the error in the logs

The logs will display which test failed and why. This is a critical skill for debugging.

Example Error Log You Will See

```
FAILED tests/test_calculator.py::TestAddition::test_add_two_positive_numbers
FAILED tests/test_calculator.py::TestAddition::test_add_negative_numbers
FAILED tests/test_calculator.py::TestAddition::test_add_positive_and_negative
AssertionError: assert -2 == 8
Where: calc.add(3, 5) returned -2 (expected 8)
4 failed, 14 passed in 0.61s
```

**Step
5**

Fix the bug and push again

Restore the add() function to a + b, commit, and push. CI will turn green again.

```
# Fix the bug in src/calculator.py
def add(self, a: float, b: float) -> float:
    return a + b # FIXED

git add src/calculator.py
git commit -m 'fix: restore correct addition logic'
git push origin main
```

7.3. Scenario 3 – Adding a New Feature with a Feature Branch

This is the real-world workflow used in professional environments. You will create a new branch, develop a feature, create a Pull Request, and watch CI run automatically:

```
# STEP 1: Create a new feature branch
git checkout -b feature/add-square-root

# STEP 2: Add sqrt function to calculator.py
# Add at the end of the Calculator class:
#
# import math (add at the top of the file)
# def sqrt(self, n: float) -> float:
#     '''Calculate square root. Raises ValueError if n < 0.'''
#     if n < 0:
#         raise ValueError('Cannot compute square root of a negative number!')
#     return math.sqrt(n)

# STEP 3: Add test cases for sqrt
# In tests/test_calculator.py, add class TestSqrt:
#
# class TestSqrt:
#     def test_sqrt_perfect_square(self, calc):
#         assert calc.sqrt(16) == 4.0
```

```
#
#     def test_sqrt_negative_raises_error(self, calc):
#         with pytest.raises(ValueError):
#             calc.sqrt(-4)

# STEP 4: Commit and push the feature branch
git add .
git commit -m 'feat: add square root function with tests'
git push origin feature/add-square-root

# STEP 5: Create a Pull Request on GitHub
# GitHub will prompt you to create a PR after pushing a new branch.
# Click 'Compare & pull request' → Enter description → Create pull request.
# CI pipeline will automatically run on this PR!
```

Why is CI on Pull Requests Important?

When CI runs on a Pull Request (before merging), it creates a critical safety net:

- Code reviewers can immediately see CI results directly on the PR page.
- GitHub can be configured to REQUIRE CI to pass before allowing a merge (Branch Protection Rules).
- If CI fails, the developer must fix it before the PR can be accepted.

This is why companies like Shopee, Tiki, and Grab Vietnam enforce this strictly: no code is merged if CI fails.

8. ADVANCED TOPICS AND REFERENCES

8.1. Advanced Features to Explore

8.1.1. Matrix Strategy – Running Tests Across Multiple Versions

Matrix strategy lets you run the same job with multiple configurations in parallel, for example testing across multiple Python versions:

```
jobs:
  test-matrix:
    runs-on: self-hosted
    strategy:
      matrix:
        python-version: ['3.9', '3.10', '3.11', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
```

```
- run: pytest tests/ -v
# GitHub creates 4 parallel jobs, one per Python version
```

8.1.2. Environment Variables and Secrets

Use Secrets to securely store sensitive information (API keys, passwords) on GitHub:

```
# Add a Secret on GitHub: Settings → Secrets and variables → Actions → New secret
# Name: DATABASE_URL | Value: postgresql://user:pass@localhost/db

# Using secrets in the workflow:
steps:
  - name: Run integration tests
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
      API_KEY: ${ secrets.API_KEY }
    run: pytest tests/integration/ -v
```

8.1.3. Scheduled Workflows – Running Automatically on a Schedule

```
on:
  schedule:
    # Run at 2:00 AM every day (UTC)
    - cron: '0 2 * * *'
    # Run at 9:00 AM every Monday
    - cron: '0 9 * * 1'
    # Cron syntax: minute hour day month weekday
    # Use cases: daily reports, database backups, checking for outdated dependencies...
```